else-if Ladder in Java

The `else-if` **ladder** is used in Java when you have multiple conditions to check. It allows you to test more than two conditions and execute different blocks of code based on the condition that evaluates to `true`. This is particularly useful when you need to check several conditions sequentially.

Syntax:

```
if (condition1) {
    // Code to be executed if condition1 is true
} else if (condition2) {
    // Code to be executed if condition2 is true
} else if (condition3) {
    // Code to be executed if condition3 is true
} else {
    // Code to be executed if none of the above conditions are true
}
```

- `condition1`, `condition2`, **etc.:** These are boolean expressions that are evaluated one by one.
- The first block of code executes when the corresponding condition is true.
- If none of the conditions are true, the `else` block is executed.

Example:

```
public class Main {
    public static void main(String[] args) {
        int number = 15;

        if (number > 20) {
            System.out.println("Number is greater than 20");
        } else if (number > 10) {
            System.out.println("Number is greater than 10 but less than or
equal to 20");
        } else if (number > 5) {
            System.out.println("Number is greater than 5 but less than or
equal to 10");
        } else {
            System.out.println("Number is 5 or less");
        }
    }
}
```

Explanation:

- **First condition:** The program checks if `number > 20`. Since `15` is not greater than 20, it moves to the next condition.
- **Second condition:** It checks if `number > 10`. Since `15` is greater than 10, this condition is true, and the program prints `"Number is greater than 10 but less than or equal to 20"`.
- If `number` were less than 10, it would continue checking the third condition and so on.

Flow of Execution:

- The program evaluates each condition sequentially. Once it finds a condition that evaluates to `true`, it executes the corresponding block of code and skips the remaining `else-if` conditions.
- If none of the `if` or `else-if` conditions are true, the `else` block is executed.

Example with Multiple Conditions:

```
public class Main {
    public static void main(String[] args) {
        int marks = 75;

        if (marks >= 90) {
            System.out.println("Grade: A");
        } else if (marks >= 80) {
            System.out.println("Grade: B");
        } else if (marks >= 70) {
            System.out.println("Grade: C");
        } else if (marks >= 60) {
            System.out.println("Grade: D");
        } else {
            System.out.println("Grade: F");
        }
    }
}
```

Explanation:

- The program checks the `marks` variable and assigns a grade based on the value:
  - If marks are `90` or more, it prints `"Grade: A"`.
  - If marks are between `80` and `89`, it prints `"Grade: B"`.
  - If marks are between `70` and `79`, it prints `"Grade: C"`.
  - If marks are between `60` and `69`, it prints `"Grade: D"`.

- If marks are less than `60`, it prints `"Grade: F"`.

In this case, if `marks = 75`, the output would be `"Grade: C"`.

Key Points of an `else-if` Ladder:

1. **Multiple Conditions:** The `else-if` ladder is useful when there are multiple conditions to be checked.
2. **Sequential Checking:** Conditions are checked one by one in the order they appear. The first `true` condition will trigger its corresponding block of code.
3. **No Fall Through:** Once a true condition is found, the program does not evaluate any further conditions.
4. **Final `else`:** The final `else` block (optional) is executed when none of the `if` or `else-if` conditions are true.
